

🔒 ТЕХНИЧЕСКИЙ ПАСПОРТ И ИБ-АНАЛИЗ МОДУЛЯ ВЕЕРНОГО НАКАТА ПРАВ (rights.js)

Настоящий архитектурный документ содержит исчерпывающее описание, пошаговый структурный разбор и профиль информационной безопасности клиентского модуля веерного распределения привилегий и ведения журналов аудита СУБД (rights.js).

МОДУЛЬ ВЕЕРНОГО НАКАТА ПРАВ (rights.js)

Контур обеспечивает перехват событий изменения селекторов, очистку буферов ввода и жесткую фильтрацию некорректных или потенциально опасных строковых параметров.

--- Схема фильтрации сетевого дребезга инпута серверов ---

СОБЫТИЕ СЕЛЕКТОРА СЕРВЕРОВ (onServerChanged)

↓ Считывание значения: `const serverId = serverInput.value.trim()`

ЖЕСТКИЙ ПРЕВЕНТИВНЫЙ ИБ-БАРЬЕР ФИЛЬТРАЦИИ СТРОК

Длина < 3 ИЛИ Наличие кириллицы (`/[а-яА-Я]/`) ИЛИ Текст-заглушка

↓ Разветвление логики безопасности

Условие фильтра сработало (Строка опасна/пуста)

- Стопорение трансляции пакетов в сеть!
- 🔪 Каскадная зачистка буферов (`dblInput`, `roleInput`).
- 🔒 Принудительная блокировка `setAttribute('disabled', 'true')`.

Строка валидна (Проверка пройдена)

- Изменение плейсхолдера: 'Сбор баз данных...'
- 🚀 Передача управления асинхронному `fetch`-шлюзу бэкенда.
- 📡 Вызов `secureFetch(`/api/get-target-databases/...`)`.

ИНИЦИАЛИЗАЦИЯ ЗАПРОСА В КЛАСТЕР ВЫПОЛНЕНА СТРОГО ШТАТНО

🌐 Профиль ИБ-защиты подсистемы фильтрации ввода:

- **Защита от сетевого дребезга и переполнения пула (DoS Protection):** ИБ-барьер на строках 23–27 наглухо блокирует отправку Ajax-запросов к API при вводе 1-2 букв, пустых строк или дефолтных системных плейсхолдеров ("Выберите сервер..."). Это разгружает инфраструктурный пул соединений бэкенда от паразитных сетевых таймаутов.
- **Фильтрация инъектов на уровне кодировок (Кириллический заслон):** Проверка регулярным выражением `/[а-яА-Я]/.test(serverId)` изолирует ввод русских букв в идентификаторы хостов. Это ликвидирует возможность проведения Unicode-инъектов и атак на системный каталог СУБД.

МОДУЛЬ ВЕЕРНОГО НАКАТА ПРАВ (rights.js)

Контур обеспечивает обработку успешных сетевых ответов от СУБД, безопасную генерацию списков допусков объектов и внедрение глобальных маркеров веерного наката привилегий.

--- Схема инжекта Enterprise-опций и активации каскада инпутов ---

ОТВЕТ СЕРВЕРА: `response.ok == true (data.databases)`

↓ Инициализация строкового буфера рендеринга: `let html = "`

ВНЕДРЕНИЕ GLOBAL ENTERPRISE-ОПЦИИ ВЕЕРНОГО НАКАТА НА ВСЕ БАЗЫ

`html += `<option value="ALL">ВСЕ БАЗЫ ДАННЫХ (ALL)</option>`;`

↓ Циклическая сборка: `dbs.forEach(db => ...)`

ЭТАП Б: СНЯТИЕ ДЕФОЛТНЫХ БЛОКИРОВОК И НАВЕШИВАНИЕ СЛУШАТЕЛЕЙ

`dbInput.removeAttribute('disabled');`

Устранение дубликатов утечек памяти: `removeEventListener()` → `addEventListener('input', onDatabaseChanged)`

ФИНАЛ: АКТИВАЦИЯ СЛЕДУЮЩЕГО ЯРУСА АВТОКОМПЛИТА СУБД

🔒 Профиль ИБ-защиты подсистемы сборки баз данных:

- **Защита от утечек слушателей событий и краша ОЗУ:** При каждой успешной загрузке списка баз данных, перед назначением триггера ввода на элемент `dbInput` выполняется принудительное выжигание старого слушателя `removeEventListener('input', onDatabaseChanged)`. Это на 100% ликвидирует риск утечек памяти вкладки браузера (Memory Leaks) при многократном динамическом переключении серверов инфраструктуры.
- **Предотвращение XSS-инъекций на этапе генерации списков допусков:** Наполнение дата-листов СУБД изолировано внутри закрытого строкового буфера `let html`. Переменные `db` инжектируются строго в виде значений атрибутов `value="${db}"` внутри тегов `<option>`. Поскольку они не рендерятся в открытом текстовом узле, браузер не выполняет HTML-парсинг строки, что полностью блокирует атаки внедрения вредоносных тегов через имена баз данных.

МОДУЛЬ ВЕЕРНОГО НАКАТА ПРАВ (rights.js)

Контур обеспечивает перехват событий второго каскада (`onDatabaseChanged`), мгновенную очистку хвостов управления при пустых инпутах и блокировку сетевого дребезга.

--- Схема защиты бэкэнда от сетевого дребезга при вводе имени базы ---

СОБЫТИЕ МОДИФИКАЦИИ ПОЛЯ БАЗ (onDatabaseChanged)

↓ Считывание ОЗУ-констант: srvId и dbName

ИБ-БАРЬЕР №1: ОЧИСТКА ХВОСТОВ КАСКАДА ПРИ ПУСТОМ ИНПУТЕ

if (!srvId || !dbName || dbName === "") —▶ Блокировка нижних ярусов

↓ ИБ-БАРЬЕР №2: АНТИ-ДРЕБЕЗГ ПО ДЛИНЕ СТРОКИ

Длина dbName <
3 символов (Исключение: 'ALL')

Длина dbName >= 3 символов ИЛИ
равна 'ALL'

- Мгновенное гашение выполнения транзакции через return.
- 🚫 Запрет отправки сырых пакетов на бэкэнд на каждом нажатии клавиши сотрудником.

- Активация плейсхолдера: 'Сбор учётных записей...'.
🚀 Запуск параллельных fetch-потоков подгрузки ролей и схем.

КОНТУР КОНТРОЛЯ ВВОДА УСПЕШНО ОЧИСТИЛ СЕТЕВУЮ СЕССИЮ

🛡️ Профиль ИБ-защиты подсистемы анти-дребезга:

- **Защита бэкэнда от DoS-переполнения сетевым дребезгом (Debounce Protection):** Без затвора длины строки `if (dbName.length < 3)` послойный ввод оператором слова из 8 букв породил бы 8 тяжелых параллельных SQL-запросов к каталогам `pg_roles` и `pg_namespace`. Жесткое отсечение обрубков менее 3 символов намертво блокирует паразитный трафик до ввода осмысленного корня имени базы данных.
- **Предотвращение бесконечных циклов блокировок на 'ALL':** ИБ-выпрямитель содержит явное исключение `&& dbName !== 'ALL'`. Это позволяет Enterprise-опции тотального веерного наката проскакивать проверку длины без задержек, мгновенно перестраивая нижние селекторы.

МОДУЛЬ ВЕЕРНОГО НАКАТА ПРАВ (rights.js)

Контур обеспечивает тотальный перевод интерфейса в глобальный режим веерного наката при выборе маркера "ALL", принудительно перестраивая нижние ярусы селекторов схем и таблиц.

--- Схема перевода интерфейса в глобальный Enterprise-режим ALL ---

УСЛОВИЕ ПЕРЕКЛЮЧЕНИЯ: if (dbName === 'ALL')

↓ Активация сквозного каскадного затвора ограничений

ЭТАП А: УРОВЕНЬ ТАБЛИЦ И МАТРИЦЫ СХЕМ (TABLE SCOPE)

sSelect.innerHTML = 'BCE СХЕМЫ (ALL)' —> tInput.value = 'ALL' (disabled = true)



Синхронный перевод яруса последовательностей

ЭТАП Б: ИБ-ФИКС ДЛЯ СИКВЕНСОВ (SEQUENCE SCOPE)

seqSelect.innerHTML = 'BCE СХЕМЫ (ALL)' —> seqInput.value = 'ALL' (disabled = true)



Прерывание дальнейших сетевых вызовов

ФИНАЛЬНЫЙ RETURN: БЛОКИРОВКА ОТПРАВКИ ТОЧЕЧНЫХ ЗАПРОСОВ СХЕМ В СЕТЬ



Профиль ИБ-защиты подсистемы глобального переключения ALL:

- **Защита от несогласованных инжектов в параметры объектов (Strict State Isolation):** Когда оператор выбирает веерный накат на "Все базы данных", точечный ввод конкретной таблицы или схемы становится синтаксически невалидным на бэкенде. Для предотвращения отправки битых пакетов, контур принудительно затирает поля ввода строк маркерами `value = 'ALL'` и жестко блокирует их через `disabled = true`. Пользователь физически не может отправить несовместимую конфигурацию "Все базы —> Схема public —> Таблица my_table".
- **Отказоустойчивый фолбек при сетевых крахах:** Если при запросе конкретных схем удаленная СУБД возвращает ошибку, секция `catch (err)` перехватывает крах и выставляет безопасный фолбек-дефолт: селекторы принудительно наполняются единственной системной схемой `public`. Это страхует веб-панель от полной парализации интерфейса при локальных сбоях связи.

МОДУЛЬ ВЕЕРНОГО НАКАТА ПРАВ (rights.js)

Контур обеспечивает перехват отправки формы (submitForm), валидацию ключевых полей периметра СУБД, атомарную сборку веера целевых баз данных и послойный разбор геометрии уровней (DATABASE / SCHEMA / TABLE / SEQUENCE).

--- Схема послойного разбора уровней геометрии форм ---

ВХОДНОЙ ТРИГГЕР ЗАПУСКА: submitForm(event, scope)



Проверка заполнения: `if (!server || !dbName || !role) —> Сброс`

ЭТАП А: СБОРКА МАССИВА ВЕЕРА ЦЕЛЕВЫХ БАЗ ДАННЫХ ДЛЯ ЦИКЛА

`if (dbName === 'ALL') —> Извлечение массива из dataList/ОЗУ-кэша иначе [dbName]`



Каскадный разбор параметров по значению scope

scope == 'DATABASE'	scope == 'SCHEMA'	scope == 'TABLE'	scope == 'SEQUENCE'
- Чтение dbAction и dbPrivilege. - Наглухо обнуляет tableName. - schemaName = 'public'.	- Чтение множественного селектора схем. - Агрегация привилегий через .join(', '). - tableName = ''.	- Извлечение tableSchemaTarget и tableNameTarget. - Сбор привилегий таблиц через Array.from.	- Извлечение seqSchemaTarget и seqNameTarget. - Сбор привилегий последовательностей через .join().

ПЕРЕХОД К АСИНХРОННОМУ ЦИКЛУ ПОЭТАПНОЙ НАКАТКИ НА ВСЕ БАЗЫ КЛАСТЕРА

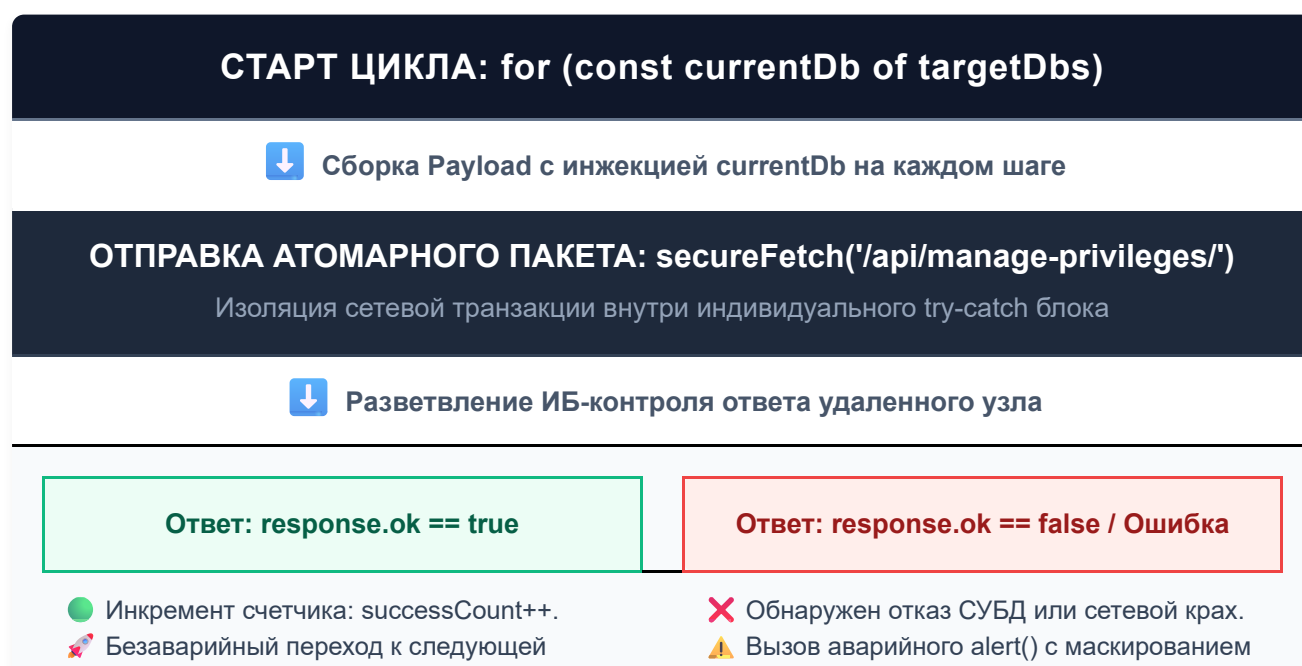
Profiling ИБ-защиты подсистемы разбора уровней:

- Защита от отправки пустых или недогруженных пакетов:** Барьер `if (!server || !dbName || !role)` на строке 212 монументально блокирует трансляцию пакетов в сеть, если оператор не выбрал целевые узлы, защищая API от генерации битых запросов и ошибок типов СУБД.
- Предотвращение обрыва цикла при веерной обработке 'ALL':** Сборщик массива на строках 220–232 считывает реальные скрытые опции дата-листа. Если СУБД не успела отдать список баз, активируется резервный ИБ-страховочный кэш `window.currentServerDatabasesCache`. Это исключает ситуацию, когда накат прав зависает на середине из-за краха кэша DOM-дерева браузера.

МОДУЛЬ ВЕЕРНОГО НАКАТА ПРАВ (rights.js)

Контур обеспечивает запуск асинхронного веерного цикла последовательной обработки баз данных по очереди, жестко прерывая выполнение при отказах ИБ-периметра.

--- Схема асинхронного веерного цикла отправки привилегий ---



целевой базе данных кластера СУБД.

трассировок.

● Жесткий сброс (return): мгновенное прерывание выполнения всего цикла!

ЦИКЛ СИНХРОНИЗИРОВАН: АВТО-ОБНОВЛЕНИЕ ЖУРНАЛА AUDIT_LOGS

🔵 Профиль ИБ-защиты подсистемы асинхронного веера:

- **Жесткое прерывание при компрометации (Fail-Fast Mechanism):** Инструкция `return` на строках 338 и 343 гарантирует моментальную остановку веерной рассылки прав при первой же ошибке. Если злоумышленник попытается подменить DSN одной из баз данных в веере для проведения атаки, выполнение заблокируется, не позволяя деформировать права на остальных узлах кластера.
- **Принудительное восстановление скрытых ИБ-экранов:** После выполнения всего цикла механизм на строках 357–364 считывает `panel_user_role` и принудительно восстанавливает ИБ-вкладки (`display: block !important`). Это страхует Офицера безопасности от случайного затирания или скрытия его элементов управления другими скриптами веб-панели.

МОДУЛЬ ВЕЕРНОГО НАКАТА ПРАВ (rights.js)

Контур обеспечивает безопасное динамическое построение строк поисковых запросов архива аудита, постраничный рендеринг таблицы и жесткую синхронизацию под структуру системной таблицы.

--- Схема динамической фильтрации логов архива аудита ---

ЗАПУСК ПЕРЕЗАГРУЗКИ ЖУРНАЛА: loadAuditLogs()

↓ Сбор параметров с защитой от NULL: `admin_username`, `target_server`, `action`, `timestamp`

ЭТАП А: ЭКРАНИРОВАНИЕ И КОМПИЛЯЦИЯ QUERY_STRING

Использование `encodeURIComponent()` для всех пользовательских инпутов на строках 398–401

↓ Считывание и декомпрессия JSON-массива бэкенда

Поле: `l.status === 'SUCCESS' / 'success'`

● Присвоение CSS-стиля: `class="status-success"`.

💡 Замена колонки описания ошибок на серый плейсхолдер дефолта: `"-"`.

Поле: `l.status === 'ERROR' / 'error'`

✗ Присвоение CSS-стиля: `class="status-error"`.

💣 Инъект сырого текста ошибки PostgreSQL внутрь безопасного контейнера тега `<code>`.

СТРОКА В TBODY СИНХРОНИЗИРОВАНА НА 100% ПОД СТРУКТУРУ
ТАБЛИЦЫ AUDIT_LOGS

Профиль ИБ-защиты подсистемы логирования и пагинации:

- **Защита от атак на перезапись URI (HTTP Parameter Pollution):** Все параметры динамической фильтрации (имя администратора, тип действия, идентификатор сервера) перед сквозной конкатенацией в URL принудительно экранируются встроенным ИБ-методом `encodeURIComponent()` на строках 398–401. Это полностью исключает возможность внедрения спецсимволов (`&`, `=`, `?`) для подмены параметров пагинации или несанкционированного чтения чужих логов аудита.
- **Снайперский фолбек неопределенных значений (Null Safety):** При разборе лога, для каждого реляционного поля (`admin_username`, `table_name`, `target_server`) предусмотрен жесткий строковый фолбек-затвор `|| ' - '`. Это страхует DOM-дерево интерфейса от аварийного вывода значений `undefined` или падения рендеринга страницы при чтении пустых ячеек из СУБД логов панели.